Name: Nour Aldeen Nabil

install kubeadm k8s cluster with 1 master and 1 worker nodes.
NOTE: REMOVE MASTERNODE TAINT TO BE ABL TO SCHEDULE PODS ON IT

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   bash - project-01

[root@master-01 project-01]# kubectl get nodes
NAME                     STATUS   ROLES           AGE   VERSION
master-01.linux.nour.local   Ready    control-plane   2d4h  v1.32.1
worker-01.linux.nour.local   Ready    <none>          2d4h  v1.32.1
[root@master-01 project-01]# kubectl get pod -n kube-system
NAME                                                     READY   STATUS    RESTARTS       AGE
coredns-668d6bf9bc-874tb                                 1/1     Running   1 (42m ago)    23h
coredns-668d6bf9bc-nlrmq                                 1/1     Running   1 (42m ago)    23h
etcd-master-01.linux.nour.local                          1/1     Running   5 (42m ago)    2d4h
kube-apiserver-master-01.linux.nour.local                1/1     Running   5 (42m ago)    2d4h
kube-controller-manager-master-01.linux.nour.local       1/1     Running   5 (42m ago)    2d4h
kube-proxy-46dk7                                         1/1     Running   7 (42m ago)    2d4h
kube-proxy-7552p                                         1/1     Running   5 (42m ago)    2d4h
kube-scheduler-master-01.linux.nour.local                1/1     Running   5 (42m ago)    2d4h
weave-net-5cz9z                                          2/2     Running   16 (39m ago)   2d4h
weave-net-fh5kk                                          2/2     Running   15 (39m ago)   2d4h
[root@master-01 project-01]#
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   bash

[root@master-01 project-01]# kubectl get svc
NAME                                          TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)                    AGE
backend                                       NodePort       10.104.136.41    <none>        3000:30008/TCP             18h
db                                            ClusterIP      10.98.228.82     <none>        3306/TCP                   18h
frontend                                      NodePort       10.101.161.162   <none>        80:30007/TCP               18h
ingress-controler-ingress-nginx-controller    LoadBalancer   10.102.99.66     <pending>     80:31911/TCP,443:30092/TCP 23h
ingress-controler-ingress-nginx-controller-admission  ClusterIP  10.97.222.219  <none>        443/TCP                    23h
kubernetes                                    ClusterIP      10.96.0.1        <none>        443/TCP                    2d4h
[root@master-01 project-01]# kubectl describe nodes master-01.linux.nour.local | grep -i taint
Taints:             <none>
[root@master-01 project-01]#
```

create a docker file for both app [frontend,backend], and docker-compose file to group the 3 tier application togther

NOTE: FRONTEND DOCKERFILE MUST BE MULTISTAGE DOCKER FILE

Run ··· project-01 [SSH: 192.168.1.150]

Dockerfile U ×

depi-angular-app > front-end > Dockerfile > ...

```
FROM node AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm install -g @angular/cli && ng build --configuration production

# Stage 2: Serve the frontend using Nginx
FROM nginx:alpine
COPY --from=build /app/dist/my-angular-app/browser/ /usr/share/nginx/html
EXPOSE 80
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
# Set the entrypoint to run the script before starting Nginx
ENTRYPOINT ["/entrypoint.sh"]
CMD ["nginx", "-g", "daemon off;"]
```

Run ··· project-01 [SSH: 192.168.1.150]

Dockerfile U ×

depi-angular-app > backend > Dockerfile > ...

```
FROM node
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Go Run Terminal Help project-01 [SSH: 192.168.1.150]

docker-compose.yml U ×

depi-angular-app > docker-compose.yml

```
version: '3.8'
Run All Services
services:
  Run Service
  frontend:
    build: ./front-end
    container_name: my_frontend
    environment:
      API_URL: http://localhost:3000 # Change this URL dynamically
    ports:
      - "80:80"
    depends_on:
      - backend

  Run Service
  backend:
    build: ./backend
    container_name: my_backend
    restart: always
    environment:
      DB_HOST: db  # Use the service name as the hostname
      DB_USER: root
      DB_PASS: rootpassword
      DB_NAME: angular
    ports:
      - "3000:3000"
    depends_on:
      - db

  Run Service
  db:
    image: mysql
    restart: always
    environment:
      MYSQL_DATABASE: angular
      MYSQL_ROOT_PASSWORD: rootpassword
    volumes:
      - sql_data:/var/lib/db/sql_data

volumes:
  sql_data:
```

push docker images to any docker registry

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS 1

[root@master-01 project-01]# docker pull localhost:5000/my-backend
Using default tag: latest
latest: Pulling from my-backend
Digest: sha256:0b41a1180b1c11fc116da867557a6dec8e73c8899a834707e1e5774a080eb59b
Status: Image is up to date for localhost:5000/my-backend:latest
localhost:5000/my-backend:latest
[root@master-01 project-01]# docker pull localhost:5000/my-frontend
Using default tag: latest
latest: Pulling from my-frontend
Digest: sha256:9c92d7d8107313e46287d3e65c9be0f5f9ec410f33ac4115723fd4324894f87f
Status: Image is up to date for localhost:5000/my-frontend:latest
localhost:5000/my-frontend:latest
[root@master-01 project-01]#
```

create kubernetes yaml files for the 3 tier application [frontend,backend,mysql] and use the docker images you have pushed
NOTE: USE PV,PVC FOR MYSQL DATA

<u>Backend</u>

```
Run   Terminal   Help                                   project-01 [SSH: 192.168.1.150]
backend.yaml ×   frontend.yaml   ingress.yaml   mysql.yaml
kubernates > backend.yaml
1  apiVersion: v1
2  kind: Pod
3  metadata:
4    creationTimestamp: null
5    labels:
6      tier: backend
7    name: backend
8  spec:
9    containers:
10   - image: 192.168.1.150:5000/my-backend
11     name: backend
12     ports:
13     - containerPort: 3000
14     env:
15     - name: DB_HOST
16       value: "db"  # Change this URL dynamically
17     - name: DB_USER
18       value: "root"  # Change this URL dynamically
19     - name: DB_PASS
20       value: "rootpassword"  # Change this URL dynamically
21     - name: DB_NAME
22       value: "angular"  # Change this URL dynamically
23
24     resources: {}
25   dnsPolicy: ClusterFirst
26   restartPolicy: Always
27 status: {}
28
```

```
29 ---
30 apiVersion: v1
31 kind: Service
32 metadata:
33   name: backend
34 spec:
35   type: NodePort
36   selector:
37     tier: backend
38   ports:
39     - protocol: TCP
40       port: 3000
41       targetPort: 3000
42       nodePort: 30008
43
44
```

Frontend

```yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    tier: frontend
  name: frontend
spec:
  containers:
  - image: 192.168.1.150:5000/my-frontend:latest
    name: frontend
    ports:
    - containerPort: 80
    env:
    - name: API_URL
      value: "http://192.168.1.100:30008"  # Change this URL dynamically
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
```

```yaml
---
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  type: NodePort
  selector:
    tier: frontend
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
    nodePort: 30007
```

DB

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mysql-pv
spec:
  capacity:
    storage: 5Gi  # Adjust size as needed
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: "/mnt/data/mysql"  # Adjust path based on your environment
```

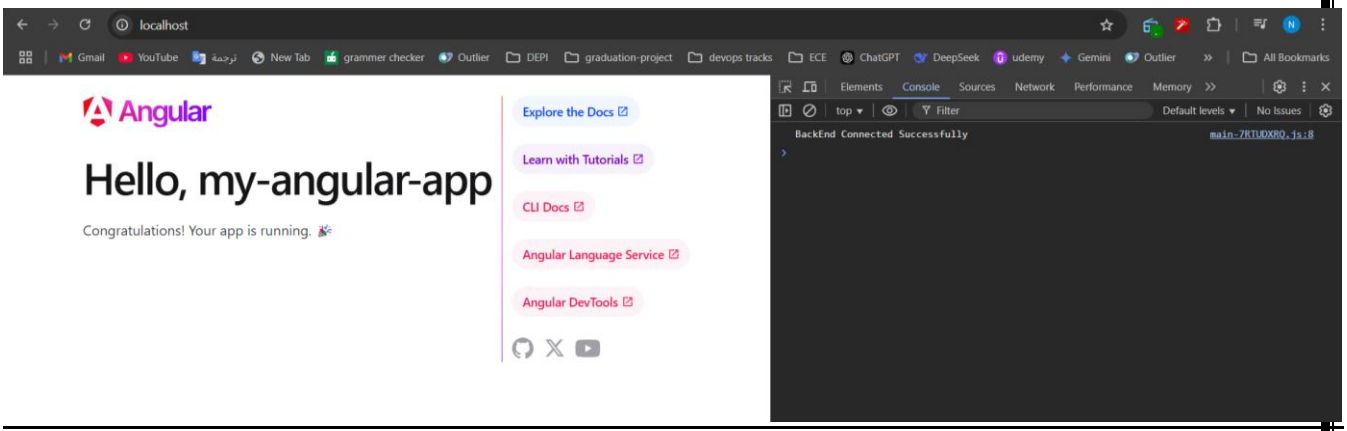
```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  storageClassName: manual
```

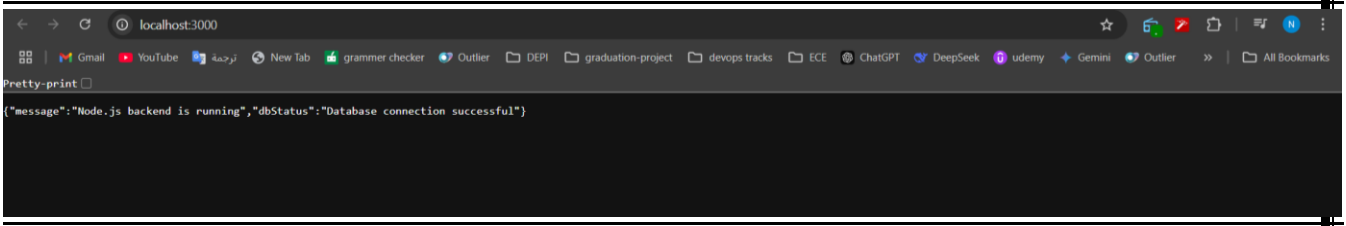
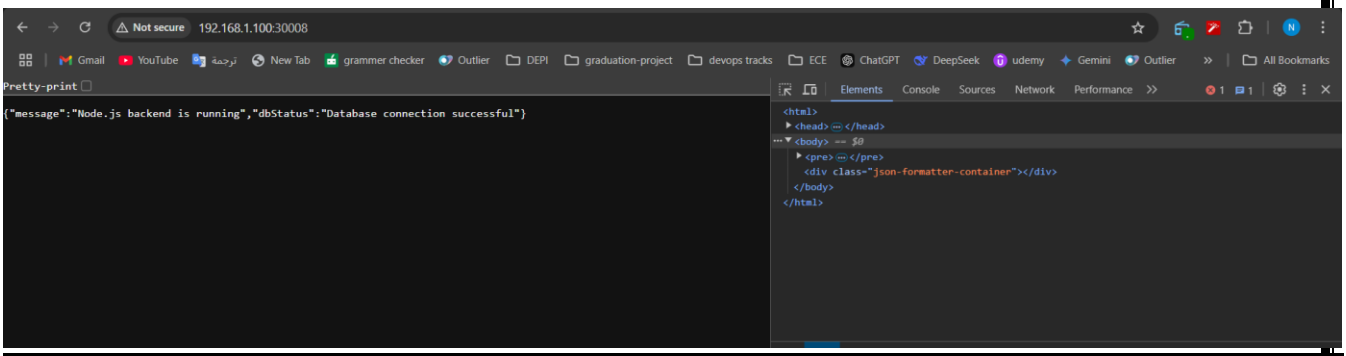
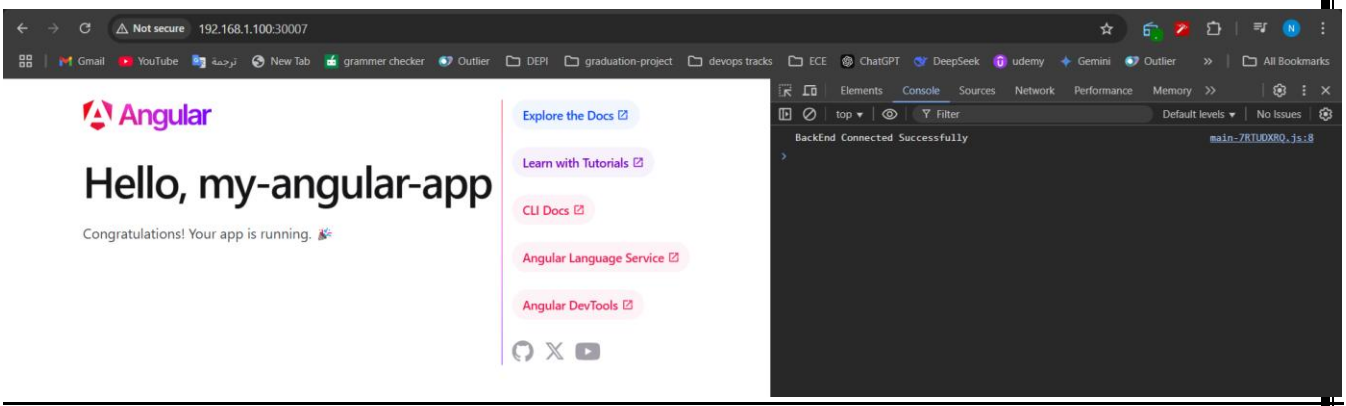
```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mysql-pod
  labels:
    tier: mysql
spec:
  containers:
  - name: db
    image: mysql
    env:
    - name: MYSQL_ROOT_PASSWORD
      value: "rootpassword"
    - name: MYSQL_DATABASE
      value: "angular"
    ports:
    - containerPort: 3306
    volumeMounts:
      - name: mysql-storage
        mountPath: /var/lib/mysql
  volumes:
  - name: mysql-storage
    persistentVolumeClaim:
      claimName: mysql-pvc
```

```yaml
---
apiVersion: v1
kind: Service
metadata:
  name: db
spec:
  selector:
    tier: mysql
  ports:
    - protocol: TCP
      port: 3306
      targetPort: 3306
```

Final verification

- **With docker compose**





- **With Kubernetes**